AI Compiler Performance Engineering

Auto-Loops Books — ch24

Contents

1 Heterogeneous Compilation and Multi-Hardware Deployment 1

1.1 Heterogeneous cluster 1
1.2 Unified stack 3
1.3 Deployment contract 4
1.4 Runtime adaptation 7
1.5 Ownership and observability 8
1.6 Version regression 10
1.7 Rollout worksheet 11
1.8 Industrial cases 14
1.9 YiRage cluster 15
1.10 Key Takeaways 16
1.11 Conclusion 17

Bibliography 18

Index 20

Chapter 1

Heterogeneous Compilation and Multi-Hardware Deployment

Production LLM serving is no longer a single-GPU problem; it is a fleet problem, where prefill and decode run on different pools, edge and datacenter run different models, and the same logical workload must compile for GPUs, CPUs, and NPUs that have nothing in common but the math (Davies *et al.*, 2025; IREE / iree-org contributors, 2024). This chapter is where every per-target decision from the MegaKernel and compiler chapters becomes a deployment-level routing decision across a heterogeneous **cluster**. The decode metrics still govern: dense Llama-3.2-1B issues 847.5 launches per token on H100 and OLMoE-1B/7B issues 9,305, and removing host overhead bought 1.259× on a Qwen-2.5-7B cell—gains that must now be preserved across a mix of silicon, not a single SKU (Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024). The mistake to avoid is deploying one hand-tuned artifact everywhere. A kernel optimal on an H100 is illegal on an XDNA tile and wasteful on a CPU, so heterogeneous deployment demands per-target compilation with a unified front end and a runtime that adapts to the hardware it finds (AMD, 2024b; Rotem *et al.*, 2018).

1.1 Heterogeneous cluster

A **hetero**geneous **cluster** exists because prefill and decode have opposite hardware appetites. Prefill is compute-bound and batches well, so it maps to dense GPU pools; decode is memory- and orchestration-bound at batch-1, so it is provisioned differently and far more heavily (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

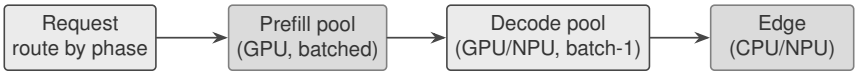


Figure 1.1: A heterogeneous serving cluster routes work by phase and target: batched prefill, batch-1 decode, and edge each map to different silicon.

Table 1.1: Roles in a heterogeneous serving cluster; each target owns a slice of traffic with a preferred compiled artifact (Davies *et al.*, 2025; Mirage Project contributors, 2025).

Hardware	Typical role	Preferred artifact
Datacenter GPU	High-throughput decode	Searched megakernel
CPU	Cost-sensitive / low-QPS	AOT bundle, cache-blocked
NPU / edge	Power-constrained	AOT bundle, static tiling

DeepSeek-V3's production layout is the canonical evidence: its minimum prefill unit spans 4 nodes (32 GPUs) while its minimum decode unit spans 40 nodes (320 GPUs), a roughly 10× capacity skew that reflects how differently the two phases use silicon (Davies *et al.*, 2025). Disaggregating them into separate pools lets each be optimized and scaled independently, which is the cluster-level form of the prefill/decode split this book has drawn since Chapter 1.

The **hetero**geneous dimension deepens when the **cluster** mixes vendor silicon. A fleet may run datacenter GPUs for high-throughput decode, CPUs for low-QPS or cost-sensitive endpoints, and NPUs for edge or power-constrained deployment, each with the distinct residency and synchronization models of Chapters 6–9 (AMD, 2024b,a; Rotem *et al.*, 2018). The deployment decision is which target each slice of traffic routes to, and the answer is workload-specific: a high-reuse decode layer justifies a searched GPU megakernel, while an edge endpoint wants a Glow-style AOT bundle (Mirage Project contributors, 2025; Rotem *et al.*, 2018). Routing across this **cluster** is the deployment-level analogue of the per-region compiler choice of Chapter 21.

The reason routing beats standardizing is the same one this book has argued since Chapter 1: the optimal schedule is hardware-specific, so a single artifact is either illegal or leaves performance on the table somewhere in the **cluster** (AMD, 2024b; SemiAnalysis, 2024). A **hetero**geneous deployment embraces this by compiling per target and routing each request to the silicon whose strengths match the work, rather than forcing a lowest-common-denominator kernel across the fleet. The routing policy itself becomes an artifact to maintain—which traffic class goes to which pool—and it shifts as models and hardware evolve, which is why encoding the rationale rather than hand-wiring the routes matters at fleet



Figure 1.2: A unified MLIR-based stack lowers one model description to per-target artifacts, sharing front-end optimizations across hardware.

scale (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

1.2 Unified stack

Heterogeneous deployment is only tractable with a **unified** compiler front end, and **mlir** has become the de facto substrate for one. By providing a common, extensible IR that many backends share, **mlir** lets a single model description carry through shared high-level optimizations before forking into per-target codegen, rather than maintaining a separate compiler per vendor (IREE / iree-org contributors, 2024; Lai *et al.*, 2023). IREE is the concrete instance: it is an **mlir**-native compiler and runtime that lowers one input through its dialect stack to llvm-cpu, CUDA, ROCm, Vulkan, Metal, and experimental AMD AIE, giving a genuinely **unified** path across the matrix (IREE / iree-org contributors, 2024). TVM's Relax and Triton's TritonGPU dialect are further evidence that the field is converging on **mlir**-style staged lowering as the way to span hardware (Lai *et al.*, 2023; Tillet *et al.*, 2019).

The value of a **unified** stack for decode specifically is that the residency, role, pipeline, and synchronization passes of Chapters 6–9 can be written once against the shared IR and specialized per target, rather than reimplemented per backend (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). This is what makes the per-target guarantees of those chapters scalable across a fleet: the same memory pass checks a 228 KB H100 budget and a 64 KB XDNA tile, the same sync pass proves deadlock-freedom on both, because they operate on one **unified** representation (NVIDIA, 2022; AMD, 2024a). Without a **unified** stack, heterogeneous deployment degenerates into a matrix of separately maintained, separately verified kernels.

It is worth being clear about what **mlir** does and does not solve. It provides the shared infrastructure—a common IR, a pass framework, reusable dialects—so that backends do not each reinvent the lowering machinery, and it lets cross-level optimizations be expressed once (Lai *et al.*, 2023; IREE / iree-org contributors, 2024). What it does not do is make the per-target decisions for free: a residency budget, a role split, and a pipeline depth still differ per device, and **mlir** only

ensures those decisions are made on a common substrate rather than in incompatible silos. The **unified** stack is therefore a force multiplier for the per-target passes, not a replacement for them—it is what lets one team maintain those passes across a growing matrix instead of one team per vendor (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

The convergence on **mlir** across TVM (Relax), Triton (TritonGPU), and IREE is itself a signal that the industry has accepted heterogeneity as permanent (Lai *et al.*, 2023; Tillet *et al.*, 2019; IREE / iree-org contributors, 2024). Rather than betting on one winning architecture, these stacks bet on a **unified** compiler layer that can absorb whatever silicon arrives, which is the only durable response to a hardware landscape that adds a new accelerator class every year (AMD, 2024a; Chen *et al.*, 2018).

1.3 Deployment contract

The useful unit of deployment is not a model file and not a container image. It is a **deployment contract**: a small, reviewable record that says which graph regions may run on which hardware, which artifact realizes each region, what runtime feature selects it, and which measurement can reject it. Treating that record as first-class is how a team prevents the familiar accident in which a GPU-only optimization is silently inherited by a CPU fallback, or an edge build is accepted even though it materializes an intermediate that the datacenter path keeps resident. The compiler has already made most of these choices by the time a request arrives; the contract makes those choices visible to the serving and release systems (IREE / iree-org contributors, 2024; Chen and YiRage Contributors, 2026; Rotem *et al.*, 2018).

For a GPU target, the contract should name the relevant compute capability, memory budget, supported asynchronous-copy or matrix instructions, allowed precision modes, and the kernel variant selected for each important shape. For a CPU target, it should instead name the ISA and vector-width assumptions, thread and NUMA policy, cache blocking assumptions, and the reference fallback. An NPU or XDNA target needs its tile-memory budget, static schedule or DMA constraints, supported operator set, and quantized layout. These are not cosmetic metadata. They are the conditions under which a compiled schedule is legal: a tile that fits in an H100 shared-memory budget need not fit on an AIE tile, and a loop ordering that is sensible for a GPU warp does not automatically expose useful SIMD reuse on a CPU (NVIDIA, 2022; AMD, 2024a,b).

The front end keeps the contract coherent by describing an operation in terms of

layouts, shapes, precision, and dependencies before it commits to a device-specific implementation. A graph split then draws the boundary between regions with different natural homes. The usual split is not "GPU versus CPU" in the abstract. It is a data-movement decision: keep a dense, high-reuse prefill region near the GPU tensor cores; retain a cache-friendly control or low-QPS path on the CPU when the transfer would dominate; emit a static NPU region only when its layout and shape constraints are satisfied. An MLIR-based stack can preserve those requirements through lowering and expose a target-specific legality failure before code generation, instead of discovering it as an opaque runtime failure (IREE / iree-org contributors, 2024; Lai *et al.*, 2023; Chen and YiRage Contributors, 2026).

This changes the release question. "Does the model run?" is a smoke test. "Does every admitted traffic class have an artifact whose assumptions match the detected hardware and whose fallback preserves correctness?" is a deployment test. The latter is what lets the same model description serve a datacenter GPU pool, a CPU pool, and an edge accelerator without reducing all three to their least capable common denominator (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018).

1.3.1 A graph-splitting decision

Graph splitting deserves an explicit rule because it is where a unified compiler can become an unhelpful abstraction. Start from the request path, not from the set of available devices. Mark the tensors that cross a proposed device boundary, their byte size, their lifetime, and whether the next consumer can use the same layout. A boundary is attractive only when the work it moves gains more from the target's compute, memory, or power characteristics than it loses in transfer, synchronization, and an additional materialized representation. For batch-1 decode, that is a demanding test: a tiny control operation may look cheap to offload, yet its synchronization can sit on the critical token path (Chen, 2026; Vellaisamy *et al.*, 2026).

Consider a layer that has a GPU-resident KV cache and a CPU-side policy decision. Moving the policy computation is reasonable only if it consumes a compact summary and returns a compact decision while leaving the KV cache in place. Moving the KV transform itself merely to "use all available hardware" is usually a warning sign, because the boundary exports a large, latency-sensitive state only to import it again. The same logic applies on an NPU. An NPU can be excellent for a static, power-constrained subgraph, but it is a poor fit for a region that depends on a dynamic layout, a device-resident cache, or a fallback that would reconstruct the state elsewhere (AMD, 2024a; Rotem *et al.*, 2018; Chen, 2026).

The compiler should therefore annotate a split with its reason: capacity, power, supported instruction, residency preservation, or operational isolation. Those reasons are more valuable than a bare placement label. When a future hardware generation changes the balance, an engineer can re-evaluate the relevant assumption instead of reverse-engineering why a graph happened to be cut at a particular node. YiRage's per-target metadata is useful in this role because it associates a backend choice with the layout, residency, and legality constraints from which the choice followed (Chen and YiRage Contributors, 2026).

1.3.2 Artifact manifests, not opaque binaries

Every deployable artifact should have a manifest alongside it. At minimum, the manifest identifies the model and compiler revision, the target triple or device family, required runtime features, input-shape and precision envelope, kernel or executable entry points, and the benchmark record that admitted it. A multi-target IREE `.vmfb` can package host instructions and device kernels in one portable container, but portability does not remove the need to record which contained variant is expected on which device (IREE / iree-org contributors, 2024). The manifest is the bridge between the compiler's internal decision and an operator's rollout decision.

The shape envelope is particularly important. A specialization may be correct for all inputs but profitable only for a narrow set of aligned dimensions or context lengths. The runtime needs permission to dispatch it only inside that envelope, and it needs a known-safe alternative outside it. Without that distinction, "fat binary" turns into an unreviewable collection of variants that appears portable until a rare request lands on the slow or unsupported path. Recording the envelope also makes performance incidents diagnosable: the operator can determine whether a tail-latency regression came from the selected hardware, an unexpected shape, or a fallback rather than guessing from aggregate throughput (IREE / iree-org contributors, 2024; Chen, 2026).

A manifest should also distinguish correctness fallback from service fallback. A correctness fallback is an implementation that returns the right result on an admitted device when the fast kernel is unavailable. A service fallback is a routing action—for example, sending a request from an unavailable edge target to a compatible remote pool—that may change latency or cost while preserving the user's request. The compiler owns the first contract; the runtime and scheduler own the second. Keeping them separate avoids falsely claiming that a CPU kernel offers the same service-level behavior as a nearby GPU simply because both execute the graph (Davies *et al.*, 2025; IREE / iree-org contributors, 2024).

1.4 Runtime adaptation

A **unified** build still has to meet the specific device it runs on, which is the job of **runtime** adaptation. The cleanest model is the ahead-of-time fat binary: IREE compiles a FlatBuffers `.vmfb` that carries host instructions and device kernels for multiple targets, and at load time the **runtime** probes the device and dispatches the matching kernel, so one artifact serves a heterogeneous fleet without a JIT (IREE / iree-org contributors, 2024). The **runtime**'s ability to **detect** the hardware—its vendor, its memory sizes, its feature set—is what lets it pick the right kernel variant, and the fat binary can carry several specializations (aligned vs unaligned shapes, large vs small matmuls) for the **runtime** to choose among (IREE / iree-org contributors, 2024).

The **detect**-and-adapt pattern also covers graceful fallback. When a preferred path is unavailable—an NPU backend that is experimental, or a GPU feature the device lacks—the **runtime** must fall back to a kernel that runs, even if slower, rather than fail (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). For decode this matters because the residency-optimal path differs per device, so the **runtime** adaptation is not cosmetic: detecting an H100 versus a smaller GPU should select a different pipeline depth and tile size, the very parameters Chapters 6–8 tuned per target (NVIDIA, 2022; Chen, 2026). Runtime adaptation is thus the deployment-time completion of compile-time per-target specialization. The division of labor is clean: the compiler decides everything it can ahead of time, per target, and the **runtime** decides only what genuinely depends on the device it lands on. For a heterogeneous fleet this keeps the per-request path thin while still letting one binary serve many targets, which is the property that makes large-scale deployment manageable rather than a per-device build explosion (IREE / iree-org contributors, 2024; Davies *et al.*, 2025).

There is a tension between adaptation and predictability that a serving team must manage. The more the **runtime** adapts—probing the device, selecting among kernel variants, falling back when needed—the more paths exist that must be tested, and the harder it is to guarantee a specific latency on a specific device (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). The AOT fat binary mitigates this by resolving most choices at build time and leaving the **runtime** only a fast **detect**-and-dispatch, so the per-request path is short and predictable, in contrast to a JIT that recompiles on first use (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). For interactive decode, where tail latency is the product metric, this argues for front-loading adaptation into the build and keeping the **runtime** decision to a quick device **detect**, not a recompilation.

1.5 Ownership and observability

Heterogeneous deployment fails when a decision has no owner. The compiler team can produce a legal GPU, CPU, and NPU artifact; the runtime team can detect a device; the serving team can route traffic; and yet the combined system can still copy a KV cache across a boundary, select a generic fallback, or admit an unsupported shape. The remedy is to assign each decision to the layer that has the information to make it, then export enough evidence that the next layer can verify it. A unified stack reduces duplicated code, but it does not eliminate this division of responsibility (IREE / iree-org contributors, 2024; Chen and YiRage Contributors, 2026).

The compiler owns graph semantics and artifact legality. It decides which operations can fuse, which layouts and precisions an executable expects, which device features it requires, and which specializations are worth emitting. It should reject a target combination whose memory, tile, ISA, or synchronization requirements cannot be met, rather than leave the runtime to discover the mismatch under live traffic. This is the compiler's version of mechanical sympathy: encode the hardware constraint close to the lowering that relies on it (NVIDIA, 2022; AMD, 2024a; Chen and YiRage Contributors, 2026).

The runtime owns selection among admitted artifacts. Given the detected device and the request shape, it chooses an executable whose manifest covers both. Its job is deliberately narrower than re-optimizing the model: it validates feature and shape predicates, binds buffers, performs a bounded dispatch, and reports the chosen path. If it falls back, it must say so in a form the serving layer can act on. A runtime that silently picks "something compatible" can preserve correctness while concealing the loss of a searched kernel, a residency property, or a latency target (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018).

The serving scheduler owns traffic-class placement. It knows the SLO, queue state, tenancy policy, locality requirements, and cost envelope that the compiler cannot see in an individual graph. It may choose a GPU pool for interactive decode and a cheaper CPU or edge path for background work, but it should only select from the runtime's admitted choices. Asking the scheduler to infer kernel legality from a device name is as fragile as asking a compiler to infer a tenant's latency promise from an IR operation (Davies *et al.*, 2025; Kwon *et al.*, 2023). The separation is healthy precisely because each layer has a different source of truth.

1.5.1 The observability record

Every request need not emit a verbose compiler trace, but every rollout needs enough observability to reconstruct the route of a representative request. The minimum useful record joins five identities: the request traffic class, the detected hardware class, the artifact-manifest revision, the selected executable or fallback, and the measurement window used to judge it. With those identities, an engineer can distinguish three very different incidents that otherwise all appear as "latency got worse": the scheduler sent a request to a different pool, the runtime selected a different variant, or the same variant regressed after a compiler or driver change (IREE / iree-org contributors, 2024; Davies *et al.*, 2025).

The record should also retain the shape and precision category rather than every raw tensor dimension. A searched megakernel may be correct across a broad shape range but only intended for a subset; an NPU bundle may be intentionally static; a CPU variant may depend on a vector-friendly alignment. Grouping requests by the manifest's declared envelope makes these changes visible without exposing request content. It lets the fleet answer a practical question during an incident: did the distribution of work leave the benchmarked envelope, or did a path inside the envelope regress? (Chen, 2026; IREE / iree-org contributors, 2024).

This is where end-to-end goodput metrics complement conventional runtime counters. Queue depth, error rate, and device utilization can show that a pool is under stress, but they cannot by themselves show whether a new artifact has increased materialized movement or launch work per generated token. Record bytes/token and launches/token for the selected path alongside latency measurements, then compare them within the same traffic and shape category. The result is an observability model that can connect a fleet symptom back to a compiler-level cause rather than merely reporting that a device is busy (Chen, 2026; Vellaisamy *et al.*, 2026).

1.5.2 An escalation path

When a target regresses, start with the manifest rather than the source code. Confirm that the expected hardware, driver, model revision, compiler revision, and shape envelope were actually selected. If any identity differs, the problem is a routing or admission issue; return it to the runtime or scheduler owner with the recorded route. If the identities match but bytes/token or launches/token changed, inspect the lowering, fusion, residency, and dispatch changes in the compiler artifact. If those metrics match but tail latency changed, the serving owner should inspect contention, queueing, and fleet placement. This sequence

avoids assigning a queueing problem to a kernel team or a backend-legality problem to an SRE (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026).

The same escalation path makes a heterogeneous fleet easier to evolve. Adding a new accelerator becomes an explicit admission exercise: define its hardware contract, compile and validate its artifacts, register its runtime detection and fallback behavior, add its rows to the regression matrix, and authorize traffic classes only after the measurements exist. Nothing in that process requires a new ad-hoc deployment model. The shared IR and manifests provide continuity, while the target-specific evidence prevents a portability claim from becoming a performance promise (IREE / iree-org contributors, 2024; Lai *et al.*, 2023; AMD, 2024a).

This is the practical meaning of multi-hardware ownership: share the model and the front-end reasoning, but localize every hardware-specific assumption, decision, and measurement. It keeps the GPU path free to use a residency-aware megakernel, gives the CPU path a deliberate cache and SIMD strategy, and lets the NPU path remain static where static scheduling is its advantage. The cluster is unified by the contract and the evidence, not by pretending its devices are interchangeable (Chen and YiRage Contributors, 2026; Rotem *et al.*, 2018; AMD, 2024b).

1.6 Version regression

Deploying across many targets multiplies the surface for **regression**, so a heterogeneous fleet needs disciplined **version** management and automated **regression** testing. A kernel **version** that improves decode on one GPU generation can regress on another, and a model or compiler **version** bump can silently change which fusion is chosen, so every target-**version** pair must be benchmarked on the same decode metrics rather than assumed equivalent (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). This is the deployment-scale form of the book's measurement discipline: track bytes/token and launches/token per target-**version**, and gate a rollout on no **regression** across the matrix.

The autobooks-style loop that produced this book is itself a model for **regression** control: a fixed harness scores every change, improvements are kept and **regression**s reverted via version control, and a results log records each experiment (Chen and YiRage Contributors, 2026). Applied to deployment, the same pattern is a **regression** matrix—each target a row, each kernel or model **version** a column, each cell a decode benchmark—that must stay green before a **version** ships. Reproducibility is the prerequisite: pinned configurations, recorded inputs,

and logged measurements, the transparency standard the fact-verification gate of this book applies to its own numbers (Chen, 2026; Goodfellow *et al.*, 2016).

The combinatorics are what make automation essential here. With several hardware targets, multiple model **version**s, and a range of decode shapes (context lengths, batch sizes), the **regression** matrix has too many cells to check by hand, so the testing must be scripted and run on every change (Davies *et al.*, 2025). The autobooks loop's keep/revert discipline scales to this directly: a change is kept only if it improves or holds the metric across the matrix, and reverted otherwise, with the results log providing an audit trail of which **version** was good on which target (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Crucially, the metric must be the decode goodput metric—bytes/token and launches/token—not a microbenchmark, because a **version** that wins an isolated GEMM can still regress end-to-end latency, the recurring trap of this book (Chen, 2026).

1.7 Rollout worksheet

A regression matrix tells a team whether an artifact is safe to promote; a rollout worksheet explains how to promote it without turning a good benchmark into a fleet incident. The worksheet begins with a traffic class, because the same artifact can be a good choice for a throughput-oriented prefill service and a bad choice for interactive decode. Record the SLO in terms the request path experiences, then record the corresponding compiler metrics: bytes moved per output token, launches or dispatches per token, and the relevant residency or synchronization condition. This keeps the release review grounded in end-to-end goodput instead of a single operator score (Chen, 2026; Vellaisamy *et al.*, 2026).

Next, fill in one row for each admitted target. The row names the device family and runtime driver, the artifact-manifest revision, the shapes exercised, the selected kernel path, and the fallback path. It also names the owner of the hardware assumption. This sounds bureaucratic until a driver update, a compiler pass change, or a model revision alters one of those assumptions. At that point the row tells the team exactly which cells require a rerun. A target matrix is not a claim that all targets perform identically; it is an explicit inventory of the conditions under which each target is allowed to serve traffic (IREE / iree-org contributors, 2024; Davies *et al.*, 2025).

The worksheet should ask four questions in this order:

1. **Is the path legal on the detected hardware?** Verify feature requirements, memory or tile budget, precision support, and graph-partition constraints

before measuring speed. A faster schedule that spills, relies on an absent instruction, or crosses an unsupported runtime boundary is not a candidate (NVIDIA, 2022; AMD, 2024a; Chen and YiRage Contributors, 2026).

2. **Does the selected artifact preserve the intended data residency?** Trace the inputs, KV state, temporary activations, and outputs through the split. If an intermediate is newly materialized or copied between devices on each token, treat it as a design change and re-measure the full request path (Chen, 2026; Kwon *et al.*, 2023).

3. **Does the measured path meet the traffic class's objective?** Compare the same context lengths and batch regime used in production, rather than borrowing a prefill-throughput result for batch-1 decode. This is where bytes/token and launches/token make a useful common language across CPU, GPU, and NPU rows (Vellaisamy *et al.*, 2026; Chen, 2026).

4. **What happens when the fast path is absent?** Exercise device detection, unsupported shapes, missing features, and the service-routing alternative. A fallback that is never tested is a second implementation with no release evidence (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018).

This ordering is deliberate. It prevents a team from spending days optimizing an artifact that cannot be safely admitted, and it stops a good GPU result from masking a broken CPU or NPU path. It also offers a practical way to decide when to stop specializing. If a target row cannot show a residency-preserving path and a traffic class that benefits from it, the honest answer may be to route that class elsewhere. Broad portability is valuable, but it is not an obligation to force every request through every accelerator (Davies *et al.*, 2025; Rotem *et al.*, 2018).

1.7.1 Canary and rollback semantics

Promotion should be monotonic in evidence. First compile and validate the artifact in isolation. Then run the full target matrix against the pinned model, runtime, and driver configuration. Then admit a narrow traffic class whose request shapes are represented in that matrix. Only after that can the scheduler widen the route. At every stage, retain the previous manifest and route as a reversible choice. The important property is not that every rollout is slow; it is that the rollback target is known, compatible with the current state, and comparable in the same metrics (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

For a GPU canary, observe whether the searched megakernel remains on its intended shape and residency path. For a CPU canary, observe whether the intended vector and cache-blocked path is selected rather than a generic scalar fallback. For an NPU canary, observe that the static bundle accepts the layout and does not divert a dynamic case into an unmeasured bridge. These are different implementation details, but their release semantics are identical: identify the selected artifact, compare it with the recorded matrix cell, and stop expansion when the selected path or the request envelope differs (IREE / iree-org contributors, 2024; AMD, 2024a; Rotem *et al.*, 2018).

Rollback should restore both the artifact and the routing decision. Reverting only the binary while leaving a scheduler route pointed at a device whose assumptions no longer hold can be as harmful as leaving the faulty binary in place. Conversely, an application-level routing rollback can protect users while compiler engineers investigate a target-specific regression. This separation lets a fleet remain available without pretending that a performance regression has been fixed merely because traffic was hidden from it (Davies *et al.*, 2025; Chen, 2026).

1.7.2 Failure modes worth rehearsing

The first failure mode is a *false-unified* build: the front end is shared, but each backend has quietly acquired divergent graph semantics, precision behavior, or fallback rules. Its symptom is that a model passes a generic test yet produces a different execution path on a particular target. Counter it with target-specific correctness tests and manifests that tie an artifact to the lowering and runtime features it requires (IREE / iree-org contributors, 2024; Lai *et al.*, 2023).

The second is a *fallback cliff*. The runtime correctly detects that a fast path is unavailable, but the fallback changes the data path so much that latency, power, or cost violates the traffic class's objective. A CPU fallback for a low-QPS endpoint can be acceptable; the same fallback on an interactive decode pool may be an incident. The contract must therefore make the fallback observable and give the scheduler a separate route when the service objective cannot be preserved (Davies *et al.*, 2025; Rotem *et al.*, 2018).

The third is a *benchmark mirage*: a new kernel improves a local operation while adding copies, launch overhead, or synchronization to the request path. The cure is not another isolated benchmark. It is a regression cell that measures the complete decode path in the representative batch regime and records the hardware and software configuration that produced the result (Vellaisamy *et al.*, 2026; Chen, 2026). This is why the matrix and the manifest belong together: the former

provides the measured outcome, while the latter explains which executable and assumptions produced it.

Finally, rehearse an artifact mismatch. Deliberately attempt to load a GPU artifact on an unsupported feature set, send an out-of-envelope shape, and remove the preferred NPU path. The expected result is not necessarily identical performance; it is a bounded, observable transition to the known correctness or service fallback. A deployment team that has practised those transitions can make hardware heterogeneity a controlled routing decision rather than a source of surprise during a model or driver release (IREE / iree-org contributors, 2024; AMD, 2024a).

1.8 Industrial cases

The clearest industrial **case** is again disaggregated **production** serving. DeepSeek-V3's separate prefill and decode pools, with the 10× capacity skew, show a **production** system mapping each phase to the hardware footprint it needs rather than forcing both onto one uniform pool (Davies *et al.*, 2025). The lesson generalizes: a **production** deployment that measures its phases separately will provision them separately, because the decode pool's bottleneck (memory and orchestration at batch-1) is different from the prefill pool's (compute throughput) (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

A second **case** class is the megakernel in **production**. Mirage Persistent Kernel compiles an LLM forward pass into a single megakernel and reports 1.2–6.7× end-to-end latency reduction over kernel-per-operator serving, a **production**-relevant result precisely because it attacks the launch storm that dominates batch-1 decode (Mirage Project contributors, 2025; Chen, 2026). A third is edge **production** on NPUs and CPUs, where Glow-style AOT bundles deploy quantized models within fixed memory budgets (Rotem *et al.*, 2018; AMD, 2024a). Each **case** is the right answer for its slice of the fleet, which is exactly why a heterogeneous deployment routes rather than standardizes.

A useful **production** pattern that ties the cases together is the cost-versus-latency frontier. A datacenter GPU pool delivers the lowest decode latency but at the highest cost per token; a CPU pool is cheaper per token at low utilization but cannot meet aggressive tail-latency targets; an edge NPU trades absolute throughput for power and locality (Davies *et al.*, 2025; Rotem *et al.*, 2018). A **production** deployment places each traffic class on the point of that frontier its service-level objective demands—interactive chat to the low-latency GPU pool, batch or background generation to cheaper hardware—and the heterogeneous

cluster is what makes that placement possible. The compiler's role is to ensure that whichever point is chosen, the artifact for it is residency-correct and decode-optimal, so the frontier is defined by genuine hardware trade-offs rather than by which target happened to get a hand-tuned kernel (Chen and YiRage Contributors, 2026; Chen, 2026).

1.9 YiRage cluster

The book's reference compiler is the cross-**cluster** router that ties these threads together. **YiRage** compiles one model description to multiple backends—CUDA, CPU, Triton, and more—and carries the Chapter 2 residency rules as checkable IR metadata, so each target gets a verified, per-hardware artifact from a single source (Chen and YiRage Contributors, 2026). Across a heterogeneous **cluster** it makes the routing decision measurable: a high-reuse decode layer on a datacenter GPU routes to a searched megakernel, an edge endpoint to an AOT bundle, a broad-portability target to whole-program scheduling, each choice backed by the same bytes/token and launches/token metrics (Mirage Project contributors, 2025; IREE / iree-org contributors, 2024; Rotem *et al.*, 2018).

What makes **YiRage** a **cluster**-level tool rather than a single-kernel one is that the residency, role, pipeline, and sync passes of Chapters 6–9 all run per target from the **unified** representation, so deploying to a new device in the **cluster** is a recompile-and-verify, not a hand-port (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). The metadata that records why a target was routed a certain way also survives model and hardware changes, so the routing decision does not have to be re-litigated each release—the deployment-scale payoff of encoding intent rather than hand-tuning (Davies *et al.*, 2025; Chen, 2026).

This closes the arc the book has traced. The residency budgets of Chapter 6, the static roles of Chapter 7, the pipelines of Chapter 8, and the synchronization of Chapter 9 were each presented as a per-target compiler pass; the compiler survey of Chapters 16–21 showed which existing stacks implement pieces of that vision; and this chapter assembles them into a fleet-level deployment where one model description compiles, via a **unified mlir** stack, to verified per-hardware artifacts that a **runtime** dispatches and a **regression** matrix guards (Chen and YiRage Contributors, 2026; IREE / iree-org contributors, 2024; Wu *et al.*, 2024). **YiRage** is the name this book gives to the compiler that does all of it together across a heterogeneous **cluster**, and the reason every earlier chapter measured against it is that it is where the book's recurring thesis—decode is bound by movement and orchestration, fixable only by residency-aware, schedule-free, per-

target compilation—is operationalized at scale (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025).

1.10 Key Takeaways

- **Disaggregate by phase across the cluster.** Prefill is compute-bound and batches; decode is memory- and orchestration-bound at batch-1, so they provision differently—DeepSeek-V3 runs a 10× larger decode pool than prefill pool (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Measure the two phases separately, and you will end up provisioning them separately (Kwon *et al.*, 2023).

- **Unify the front end with MLIR.** A shared MLIR-based IR (IREE, Relax, TritonGPU) lets one model carry shared optimizations before per-target codegen, so the residency/role/pipeline/sync passes are written once and specialized per target (IREE / iree-org contributors, 2024; Lai *et al.*, 2023). Without it, deployment becomes an unwieldy matrix of hand-maintained kernels (Chen and YiRage Contributors, 2026).

- **Adapt at runtime by detecting the device.** A fat binary carrying multiple target kernels lets the runtime probe the hardware and dispatch the right variant, with graceful fallback when a path is unavailable (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). Detected device sizes should select pipeline depth and tile size (NVIDIA, 2022).

- **Gate rollouts on a per-target regression matrix.** A kernel or model version that helps one target can regress another, so benchmark every target-version pair on bytes/token and launches/token and require no regression before shipping (Vellaisamy *et al.*, 2026; Chen, 2026). Reproducibility (pinned configs, logged measurements) is the prerequisite (Goodfellow *et al.*, 2016).

- **Route, don't standardize.** Each production case—searched megakernel on GPU (1.2–6.7×), AOT bundle on edge, whole-program scheduling for breadth—owns a slice of the fleet, and YiRage routes per slice with verified per-target artifacts (Mirage Project contributors, 2025; Rotem *et al.*, 2018; Chen and YiRage Contributors, 2026). One artifact everywhere is the anti-pattern, because a kernel optimal on one target is illegal or wasteful on another (AMD, 2024b).

1.11 Conclusion

Heterogeneous deployment is where the book's per-target discipline becomes a fleet-level routing problem: disaggregate prefill and decode, unify the front end with MLIR so the residency, role, pipeline, and synchronization passes run once and specialize per target, adapt at runtime by detecting the device, and gate every rollout on a per-target regression matrix (Davies *et al.*, 2025; IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). The YiRage compiler is the cross-cluster router that makes this measurable, compiling one model to verified per-hardware artifacts and recording the routing intent as metadata so it survives change (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). Deployment, however, is increasingly something compilers do for themselves: Chapter **??** turns to auto-optimization and AI-assisted compilation, where the search and routing decisions of this book are themselves driven by learned and automated systems (Wu *et al.*, 2024; Chen, 2026).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 2, 3, 4, 5, 8, 10, 12, 13, 14

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 1, 2, 4, 10, 16

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17

Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Shen, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018). TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594. Graph- and operator-level optimization, operator fusion, memory latency hiding, ML-based cost model; CPU/GPU/FPGA. 4

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 3, 4, 5, 6, 8, 10, 11, 12, 15, 16, 17

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 11, 16

IREE / iree-org contributors (2024). IREE: Intermediate representation execution environment — an MLIR-based end-to-end compiler and runtime. `https://iree.dev`. AOT MLIR compiler+runtime; Flow/Stream/HAL/VM dialects; dispatch-region fusion; `.vmfb` fat binary; targets llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, Metal, AMD AIE (experimental). 1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 8, 12, 14, 16

Lai, R., Shao, J., Feng, S., Jin, Y., *et al.* (2023). Relax: Composable abstractions for end-to-end dynamic machine learning. *arXiv preprint arXiv:2311.02103*. TVM Unity; cross-level IRModule (Relax graph + TensorIR), first-class symbolic shapes, call_tir DPS; replaces Relay. 3, 4, 5, 10, 13, 16

Mirage Project contributors (2025). Mirage persistent kernel (MPK): A compiler and runtime for mega-kernelizing tensor programs. `https://github.com/mirage-project/mirage`. Transforms LLM inference into a single megakernel; SM-level task graph (tGraph); cross-operator pipelining; 1.2–6.7× latency reduction. 2, 14, 15, 16, 17

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 3, 4, 7, 8, 12, 16

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1

Rotem, N., Fix, J., Abdulrasool, S., Catron, G., Deng, S., Dzhabarov, R., Gibson, N., Hegeman, J., Lele, M., Levenstein, R., Montgomery, J., Maher, B., Nadathur, S., Olesen, J., Park, J., Rakhov, A., and Smelyanskiy, M. (2018). Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*. Two-phase strongly-typed IR (high-level graph + low-level address-only); lowering to linear-algebra primitives; profile-guided quantization; AOT bundles. 1, 2, 4, 5, 7, 8, 10, 12, 13, 14, 15, 16

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 2

Tillet, P., Kung, H. T., and Cox, D. (2019). Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL@PLDI)*, pages 10–19. Tile = statically shaped sub-array; Triton-C + LLVM-based Triton-IR; matmul/conv on par with cuBLAS/cuDNN. 3, 4

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 5, 9, 10, 11, 12, 13, 14, 16, 17

Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. (2024). Mirage: A multi-level superoptimizer for tensor programs. *arXiv preprint arXiv:2405.05751*. μGraph kernel/block/thread levels; joint algebraic+schedule+custom-kernel superoptimization; abstraction-based pruning; probabilistic equivalence via PIT over finite fields (LAX); 1.1–2.9×. 3, 4, 15, 17

Index

Heterogeneous deployment, 1

Multi-hardware, 1